TEXT MINING PARA BUSINESS ANALYTICS

Análise de Sentimentos em Dados de Saúde Mental

De modelos clássicos (TF-IDF) ao fine-tuning de Transformers (DistilBERT), com Explainable AI

João Coelho
Mestrado em Business Intelligence and Analytics

Introdução, Objetivos e Trabalho Relacionado

Enquadramento e objetivos

- Análise de sentimentos como tarefa de classificação supervisionada da polaridade emocional do texto.
- Dataset original com 3 classes; remove-se a classe neutral para uma classificação binária positivo / negativo, reduzindo ambiguidade.
- Objetivo: pipeline completo de Text Mining comparando modelos clássicos com um LLM.
- Camada extra de interpretabilidade e validação metodológica (XAI, significância, robustez).

Trabalho relacionado

Obagbuwa et al. (2023) Deteção de depressão no Twitter — LR e SVM com melhor desempenho.

Sankar et al. (2024) BoW vs TF-IDF; SVM + TF-IDF ~90% de accuracy.

Aizawa et al. (2023) BERT em tweets de saúde mental, accuracy 81%.

IEEE (2024) 53.000+ publicações; LR + TF-IDF, accuracy 87%.

Lossio-Ventura et al. (2024) LLMs eficazes na análise de texto clínico.

Visão geral do pipeline



A divisão treino/teste é feita logo a seguir ao pré-processamento — toda a aprendizagem (TF-IDF, χ^2, oversampling) usa apenas o treino, evitando data leakage.

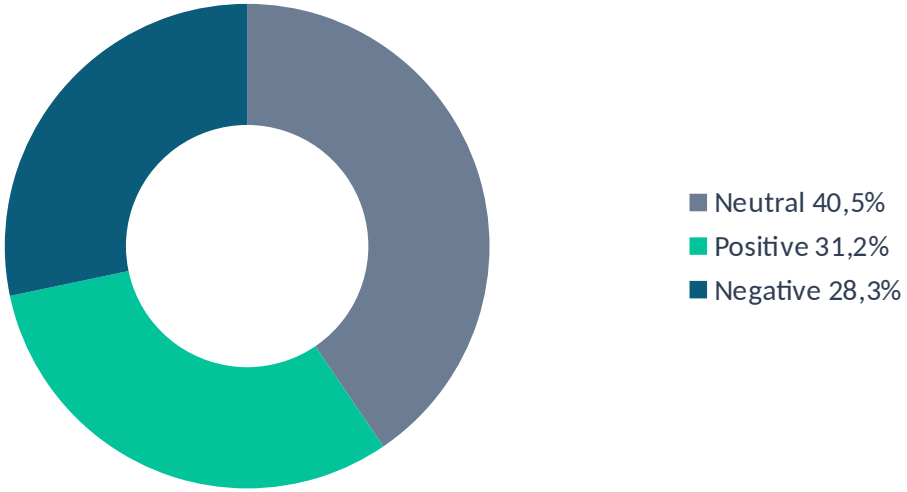
Dataset e Análise Exploratória

27 481
observações iniciais

1
único valor nulo

5–20
palavras por texto

Distribuição das classes (dataset original)



Principais observações

- Textos curtos e informais — cada palavra é discriminativa.
- Termos mais frequentes: day, good, work, time, today, love.
- Faixas etárias equilibradas (~4 580 observações por grupo).
- Desequilíbrio inicial justifica a passagem a problema binário.

Pré-processamento Avançado

Após remover
a classe neutral

16 363

observações

8 582
exemplos positivos

7 781
exemplos negativos

Pipeline de transformações textuais

1. Remoção de HTML e URLs
2. Expansão de chatwords
3. Expansão de emoticons
4. Conversão de emojis em texto
5. Conversão para minúsculas
6. Expansão de contrações
7. Remoção de pontuação
8. Remoção de stopwords
9. Filtragem de palavras válidas
10. Lematização
11. Identificação de entidades (NER)

→ **coluna text_preprocessed**

Representação e Seleção de Atributos

Prevenção de data leakage: divisão treino/teste feita ANTES da vetorização, da seleção de atributos e do oversampling.

1

70 / 30

Divisão estratificada

Treino e teste separados logo no início.

→

2

TF-IDF

Vetorização

fit no treino, transform no teste.

→

3

χ^2

Seleção (SelectKBest)

Atributos escolhidos só com rótulos de treino.

→

4

Oversampling

Balanceamento

Aplicado apenas ao conjunto de treino.

O conjunto de teste mantém a distribuição real dos dados — avaliação independente e fiável.

Implementação — pipeline sem data leakage

```
# 1) Divisão treino/teste ANTES de tudo
text_train, text_test, y_train, y_test = train_test_split(
    review_df['text_preprocessed'], review_df[TARGET_COL],
    test_size=0.30, random_state=0,
    stratify=review_df[TARGET_COL])

# 2) TF-IDF: fit só no treino, transform no teste
vectorizer = TfidfVectorizer(max_features=5000)
X_train = vectorizer.fit_transform(text_train)
X_test  = vectorizer.transform(text_test)

# 3) Seleção χ² — usa apenas rótulos de treino
selector = SelectKBest(chi2, k=1000).fit(X_train, y_train)

# 4) Oversampling — aplicado SÓ ao treino
X_train, y_train = RandomOverSampler(random_state=42)
                   .fit_resample(X_train, y_train)
```

1 — Divisão primeiro
Treino e teste separados antes de qualquer ajuste.

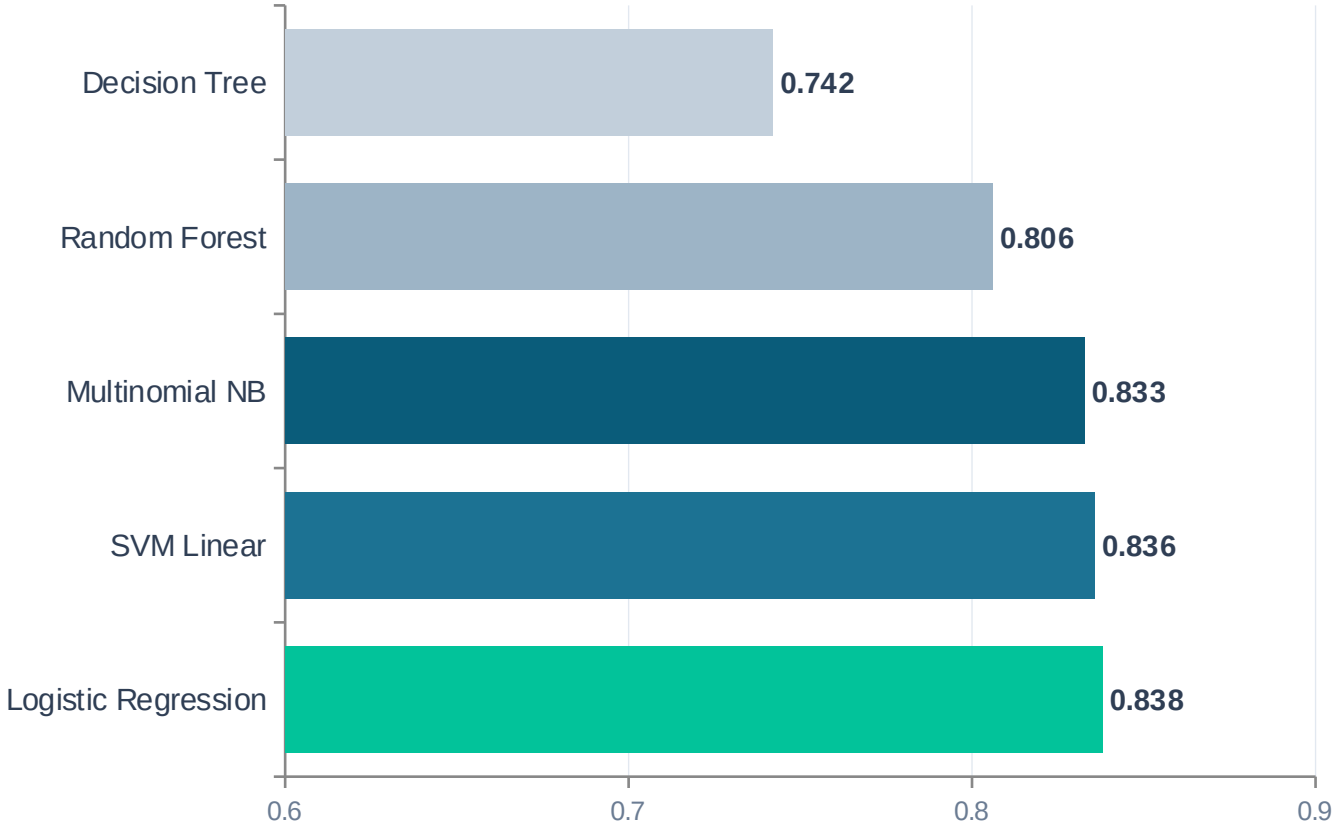
2 — TF-IDF
fit só no treino; o teste é apenas transformado.

3 — Seleção χ^2
Features escolhidas com os rótulos de treino.

4 — Oversampling
Só no treino — o teste nunca é duplicado.

Comparação de Classificadores Clássicos

F1-score no conjunto de teste



Leitura

- Ordenação por F1-score, depois Kappa e accuracy de validação cruzada.
- Logistic Regression lidera (0,8379), seguida de SVM Linear (0,8357) e Naive Bayes (0,8327).
- Modelos lineares destacam-se em matrizes TF-IDF esparsas de alta dimensão.
- Árvores e ensembles não superaram os modelos lineares.
- Elementos extra: N-grams (uni + bigramas) e otimização com GridSearchCV.

Matriz de confusão — Logistic Regression



Conjunto de teste: 4 909 exemplos reais

Métricas derivadas

Accuracy	**0,835**
Precision	**0,866**
Recall	**0,812**
Especificidade	**0,861**
F1-score	**0,838**

Leitura

Os erros repartem-se por 325 falsos positivos e 484 falsos negativos. O modelo é mais conservador a prever “positivo” — daí o recall (0,812) ser inferior à precision (0,866).

Melhor Modelo Clássico vs LLM

Logistic Regression

Melhor modelo clássico (TF-IDF)

Accuracy	0,8352
Precision	0,8655
Recall	0,8120
F1-score	0,8379
Kappa	0,6707

DistilBERT (fine-tuned)

Texto original — melhor desempenho global

Accuracy	0,9000
Precision	0,9638
Recall	0,8313
F1-score	0,8926
Kappa	0,8000

- Texto original supera o pré-processado (F1 0,8682; Kappa 0,7414).
- Pré-processamento ajuda TF-IDF mas remove contexto útil ao LLM.
- Trade-off: melhor desempenho vs maior custo computacional.

DistilBERT vence em accuracy, precision, F1 e Kappa — captura semântica contextual que o TF-IDF não modela.

Exemplos em frases novas — Clássico vs LLM

Concordância em 10 de 12 comentários. As duas divergências são casos em que o DistilBERT capta melhor o contexto.

Comentário	Logistic Reg.	DistilBERT
“I feel much better today and I am proud of the progress I made.”	positive	positive
“I am exhausted and I do not know how to keep going anymore.”	negative	negative
★ “Talking to someone really helped me and made me feel supported.”	negative	positive
★ “I am grateful for the support I received from my friends today.”	negative	positive
“I could not sleep all night and my anxiety is getting worse.”	negative	negative
“Small progress is still progress, and that makes me happy.”	positive	positive

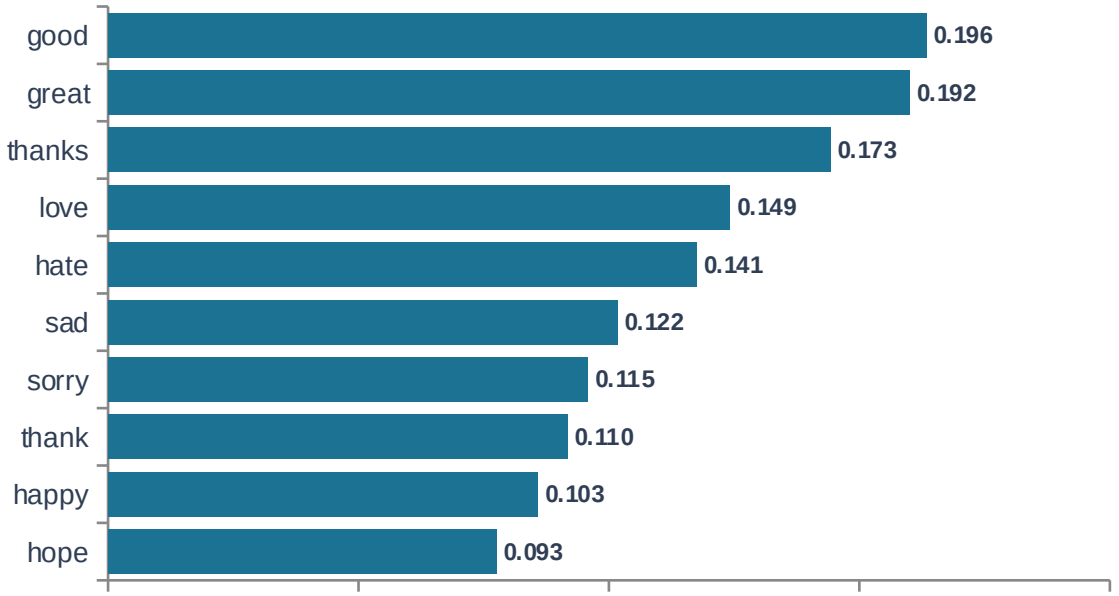
★ Nestes dois casos, o modelo clássico (saco de palavras) vê termos isolados e falha; o DistilBERT percebe que “helped me” e a gratidão indicam evolução positiva.

Explainable AI com SHAP

Modelo interpretado: Logistic Regression (TF-IDF) · 200 amostras × 1000 features · valores SHAP face à classe **positive**

Explicabilidade global — palavras mais influentes

Valor médio absoluto de SHAP (Top 10)

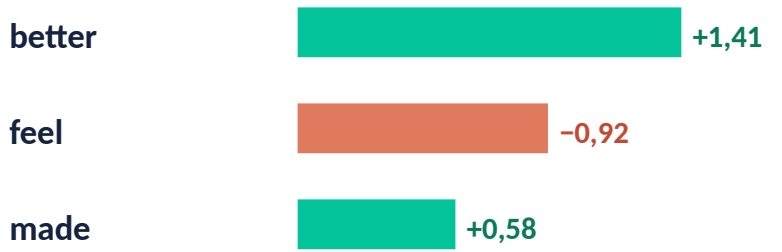


Explicabilidade local — exemplo concreto

"I feel much better today and I am proud of the progress I made."

Previsão: **positive** · confiança 0,637

Contribuição de cada palavra (SHAP)



Verde empurra para positive · **vermelho** para negative

Significância Estatística e Robustez

Teste de McNemar

320 exemplos comuns

0,9000
DistilBERT

0,8875
Logistic Regression

p-value = 0,6434 (> 0,05)

Apesar da vantagem numérica do DistilBERT, a diferença não é estatisticamente significativa nesta amostra.

Robustez por subgrupos

Accuracy global do teste dedicado: 0,8318

Por faixa etária

0,8203 – 0,8424

Estável

Por período do dia

0,8222 – 0,8422

Estável

Por país

0,7353 – 0,9355

Cautela (amostras pequenas)

Conclusões e Limitações

Melhor modelo clássico

Logistic Regression F1 0,8379

TF-IDF + classificador linear, eficaz e leve.

Melhor modelo global

DistilBERT Acc 0,9000 · F1 0,8926

Texto original; capta semântica contextual.

Síntese

- Pipeline completo e metodologicamente rigoroso, com correção do data leakage.
- Comparação estruturada de classificadores e fine-tuning de um Transformer.
- XAI, significância estatística (McNemar) e robustez por subgrupos.

Limitações

- Remoção da classe neutral simplifica a tarefa original.
- Oversampling pode repetir padrões (aplicado só ao treino).
- Custo computacional dos Transformers; SHAP no modelo clássico.

Apêndice — excertos de código

Slide de apoio para perguntas técnicas

1 · Limpeza de texto (excerto)

```
def clean_text(text):
    text = str(text).lower()
    text = re.sub(r'http\S+|www\S+', '', text)
    text = re.sub(r'\@\w+|\#', '', text)
    text = re.sub(r'[^\w\s]', '', text)
    return ' '.join(w for w in text.split()
                    if w not in stop_words)
```

2 · Reprodutibilidade do DistilBERT

```
SEED = 42
set_seed(SEED)   # antes de cada from_pretrained
args = TrainingArguments(
    seed=SEED, data_seed=SEED,
    learning_rate=2e-5, num_train_epochs=2)
```

3 · Teste de significância (McNemar)

```
# statsmodels
from statsmodels.stats
     .contingency_tables import mcnemar

# tabela 2x2 de concordância
# entre os dois modelos
res = mcnemar(tabela,
        exact=False, correction=True)
print(res.statistic, res.pvalue)
# -> p-value = 0.6434  (nao significativo)
```

Porquê estes três?

- Limpeza: base de todo o pipeline de texto.
- set_seed + data_seed: resultados do LLM reproduzíveis.
- McNemar: comparação honesta entre modelos.